See the Assessment Guide for information on how to interpret this report.

ASSESSMENT SUMMARY

```
Compilation:  PASSED
API:          PASSED

SpotBugs:     FAILED (1 warning)
PMD:          PASSED
Checkstyle:   FAILED (0 errors, 3 warnings)

Correctness:  25/28 tests passed
Memory:       No tests available for autograding.
Timing:       No tests available for autograding.

Aggregate score: 90.36%
[ Compilation: 5%, API: 5%, Style: 0%, Correctness: 90% ]
```

ASSESSMENT DETAILS

```
The following files were submitted:
----------------------------------
1.5K Oct 29 17:23 Birthday.java
1.1K Oct 29 17:23 DiscreteDistribution.java
2.6K Oct 29 17:23 Minesweeper.java
1.2K Oct 29 17:23 ThueMorse.java


================================================================================
*  COMPILING
================================================================================


% javac DiscreteDistribution.java
*------------------------------------------------------------

% javac ThueMorse.java
*------------------------------------------------------------

% javac Birthday.java
*------------------------------------------------------------

% javac Minesweeper.java
*------------------------------------------------------------


================================================================

Checking the APIs of your programs.
*------------------------------------------------------------
DiscreteDistribution:

ThueMorse:

Birthday:

Minesweeper:

================================================================


================================================================================
*  CHECKING STYLE AND COMMON BUG PATTERNS
================================================================================


% spotbugs *.class
*------------------------------------------------------------
H D NS_DANGEROUS_NON_SHORT_CIRCUIT NS: Questionable use of a non-short-circuit logic operator ('&' or '|'). Did you mean to use '&&' or '||' instead?
SpotBugs ends with 1 warning.


================================================================


% pmd .
*------------------------------------------------------------


================================================================


% checkstyle *.java
*------------------------------------------------------------
[WARN] DiscreteDistribution.java:13:15: The local variable 'S' must start with a lowercase letter and use camelCase. [LocalVariableName]
[WARN] Minesweeper.java:41:18: The local variable 'M' must start with a lowercase letter and use camelCase. [LocalVariableName]
Checkstyle ends with 0 errors and 2 warnings.

% custom checkstyle checks for DiscreteDistribution.java
*------------------------------------------------------------

% custom checkstyle checks for ThueMorse.java
*------------------------------------------------------------
```

```
% custom checkstyle checks for Birthday.java
*-----------------------------------------------------------

% custom checkstyle checks for Minesweeper.java
*-----------------------------------------------------------
[WARN] Minesweeper.java:27: Calling 'Math.random()' in more than one place suggests poor design in this program. [Design]
Checkstyle ends with 0 errors and 1 warning.


================================================================


********************************************************************************
*  TESTING CORRECTNESS
********************************************************************************

Testing correctness of DiscreteDistribution
*-----------------------------------------------------------
Running 6 total tests.

Test 1: check output format
  % java DiscreteDistribution 9 1 1 1 1 1 1
  6 5 5 3 5 4 5 3 6

  % java DiscreteDistribution 8 10 20 30 40 50 60 50 40 30 20 10
  9 5 5 2 5 7 5 4

  % java DiscreteDistribution 7 10 10 10 10 10 50
  4 3 6 3 2 1 6

  % java DiscreteDistribution 6 50 50
  2 2 1 1 1 2

  % java DiscreteDistribution 5 80 20
  2 1 1 1 2

  % java DiscreteDistribution 4 301 176 125 97 79 67 58 51 46
  5 9 1 3

  % java DiscreteDistribution 3 19 49 60 47 32 18 3 3 1
  2 3 2

  % java DiscreteDistribution 2 9316001 10274874 10109130 10045436 9850199 6704495 5886889
  2 7

  % java DiscreteDistribution 1 8167 1492 2782 4253 12702 2228 2015 6094 6966 153 772     ...
  18

==> passed

Test 2: check that output contains correct number of integers
  * fair die                          [ repeated 1000 times ]
  * sum of two dice                   [ repeated 1000 times ]
  * loaded die                        [ repeated 1000 times ]
  * fair coin                         [ repeated 1000 times ]
  * 80/20 biased coin                 [ repeated 1000 times ]
  * 9 digits in Benford's law         [ repeated 1000 times ]
  * goals in FIFA World Cup 1990-2002  [ repeated 1000 times ]
  * U.S. birthdays by day of week     [ repeated 1000 times ]
  * 26 letters in English language    [ repeated 1000 times ]
==> passed

Test 3: check that output is a sequence of integers between 1 and n
  * fair die                          [ repeated 1000 times ]
  * sum of two dice                   [ repeated 1000 times ]
  * loaded die                        [ repeated 1000 times ]
  * fair coin                         [ repeated 1000 times ]
  * 80/20 biased coin                 [ repeated 1000 times ]
  * 9 digits in Benford's law         [ repeated 1000 times ]
  * goals in FIFA World Cup 1990-2002  [ repeated 1000 times ]
  * U.S. birthdays by day of week     [ repeated 1000 times ]
  * 26 letters in English language    [ repeated 1000 times ]
==> passed

Test 4: check that program produces different results when run twice
  * fair die                          [ repeated 10 times ]
  * sum of two dice                   [ repeated 12 times ]
  * loaded die                        [ repeated 10 times ]
  * fair coin                         [ repeated 20 times ]
  * 80/20 biased coin                 [ repeated 30 times ]
  * 9 digits in Benford's law         [ repeated 10 times ]
  * goals in FIFA World Cup 1990-2002  [ repeated 10 times ]
  * U.S. birthdays by day of week     [ repeated 14 times ]
  * 26 letters in English language    [ repeated 10 times ]
==> passed

Test 5: check randomness
  * fair die                          [ repeated 100000 times ]
    - command-line arguments = "100000 1 1 1 1 1 1"

           value  observed  expected   2*O*ln(O/E)
       ----------------------------------------
               1      9982   16666.7     -10234.09
               2     20056   16666.7       7425.44
               3     20062   16666.7       7439.66
               4     19943   16666.7       7158.24
               5     20152   16666.7       7653.44
               6      9805   16666.7     -10403.46
       ----------------------------------------
                  100000  100000.0       9039.23

    G-statistic = 9039.23 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.
```

```
  * sum of two dice                    [ repeated 100000 times ]
  * loaded die                         [ repeated 100000 times ]
  * fair coin                          [ repeated 100000 times ]
  * 80/20 biased coin                  [ repeated 100000 times ]
  * 9 digits in Benford's law          [ repeated 100000 times ]
  * 26 letters in English language     [ repeated 100000 times ]
  * goals in FIFA World Cup 1990-2002  [ repeated 100000 times ]
    - command-line arguments = "100000 19 49 60 47 32 18 3 3 1"

          value  observed  expected   2*O*ln(O/E)
      ----------------------------------------------
              1      8099    8189.7       -180.30
              2     21311   21120.7        382.33
              3     25960   25862.1        196.23
              4     20122   20258.6       -272.32
              5     13960   13793.1        335.80
              6      7752    7758.6        -13.24
              7      1272    1293.1        -41.86
              8      1318    1293.1         50.27
              9       206     431.0       -304.18
      ----------------------------------------------
                  100000  100000.0        152.73

    G-statistic = 152.73 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

  * U.S. birthdays by day of week      [ repeated 100000 times ]
==> FAILED

Test 6: check randomness when n = 1
  * a_1 = 1                            [ repeated 100000 times ]
  * a_1 = 100                          [ repeated 100000 times ]
==> passed


DiscreteDistribution Total: 5/6 tests passed!


*----------------------------------------------------------
Running 5 total tests.

Test 1: check output format
  % java ThueMorse 2
  +  -
  -  +

  % java ThueMorse 4
  +  -  -  +
  -  +  +  -
  -  +  +  -
  +  -  -  +

  % java ThueMorse 8
  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +
  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +
  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +

  % java ThueMorse 16
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  -  +  +  -  +  -  -  +  +  -  -  +  -  +  +  -
  +  -  -  +  -  +  +  -  -  +  +  -  +  -  -  +

==> passed

Test 2: check correctness when n is a power of 2
  * java ThueMorse 2
  * java ThueMorse 4
  * java ThueMorse 8
  * java ThueMorse 16
  * java ThueMorse 32
  * java ThueMorse 64
==> passed

Test 3: check correctness when n is not a power of 2
  * java ThueMorse 3
  * java ThueMorse 5
  * java ThueMorse 6
  * java ThueMorse 7
  * java ThueMorse 9
  * java ThueMorse 10
  * java ThueMorse 11
  * java ThueMorse 12
  * java ThueMorse 13
```

```
   * java ThueMorse 14
   * java ThueMorse 15
==> passed

Test 4: check corner case
   * java ThueMorse 1
==> passed

Test 5: check random values of n
   * 100 random values of n in [16, 32)
   * 100 random values of n in [32, 64)
   * 50 random values of n in [64, 128)
   * 25 random values of n in [128, 256)
==> passed


ThueMorse Total: 5/5 tests passed!


================================================================
Testing correctness of Birthday
*----------------------------------------------------------
Running 6 total tests.

Test 1: check output format
  % java Birthday 365 100000
  1 0 0.0
  2 270 0.0027
  3 535 0.00805
  4 743 0.01548
  5 1065 0.02613
  6 1370 0.03983
  7 1575 0.05558
  8 1826 0.07384
  9 2039 0.09423
  10 2235 0.11658
  11 2378 0.14036
  12 2589 0.16625
  13 2743 0.19368
  14 2906 0.22274
  15 2959 0.25233
  16 3078 0.28311
  17 3182 0.31493
  18 3120 0.34613
  19 3221 0.37834
  20 3249 0.41083
  21 3308 0.44391
  22 3335 0.47726
  23 3171 0.50897

  % java Birthday 31 100000
  1 0 0.0
  2 3245 0.03245
  3 6309 0.09554
  4 8985 0.18539
  5 10665 0.29204
  6 11615 0.40819
  7 11640 0.52459

==> passed

Test 2: check values in first column
   * java Birthday 365 10000
   * java Birthday 31 10000
   * java Birthday 1 1000
   * java Birthday 2 1000
==> passed

Test 3: check that cumulative percentages are monotone nondecreasing
        and table stops when percentage reaches (or exceeds) 50%
   * java Birthday 365 10000 [ repeated 10 times ]
   * java Birthday 31 10000 [ repeated 10 times ]
   * java Birthday 10 5 [ repeated 1000 times ]
   * java Birthday 4 4 [ repeated 1000 times ]
   * java Birthday 2 2 [ repeated 1000 times ]
==> passed

Test 4: check that cumulative percentages are consistent with frequencies
   * java Birthday 365 10000
   * java Birthday 31 10000
==> passed

Test 5: check that each execution of program outputs a different table
   * java Birthday 365 10000 [ repeated twice ]
   * java Birthday 31 10000 [ repeated twice ]
==> passed

Test 6: check randomness of birthdays
   * java Birthday 365 1000000
   * java Birthday 31 1000000

            value  observed  expected   2*O*ln(O/E)
        ----------------------------------------------
               2     32773   32258.1       1038.05
               3     63884   62435.0       2931.45
               4     89098   87610.4       3000.41
               5    106282  105509.2       1551.16
               6    115779  114868.9       1827.33
            >= 7    592184  597318.4     -10224.63
        ----------------------------------------------
                   1000000 1000000.0        123.77

    G-statistic = 123.77 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.
```

```
  * java Birthday 7 1000000

          value  observed  expected   2*O*ln(O/E)
      --------------------------------------------
              2    153096  142857.1      21194.62
              3    256257  244898.0      23236.98
          >= 4    590647  612244.9     -42424.78
      --------------------------------------------
                  1000000 1000000.0       2006.83

    G-statistic = 2006.83 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

  * java Birthday 5 1000000

          value  observed  expected   2*O*ln(O/E)
      --------------------------------------------
              2    218744  200000.0      39192.24
          >= 3    781256  800000.0     -37045.36
      --------------------------------------------
                  1000000 1000000.0       2146.89

    G-statistic = 2146.89 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

==> FAILED


Birthday Total: 5/6 tests passed!


================================================================
Testing correctness of Minesweeper
*----------------------------------------------------------
Running 11 total tests.

Test 1: check output format
  % java Minesweeper 9 9 10
  0  0  0  1  *  1  0  1  *
  1  1  0  2  2  2  1  2  2
  *  2  1  2  *  1  1  *  1
  *  2  1  *  2  2  2  2  1
  1  1  1  1  2  2  *  1  0
  0  0  0  0  1  *  2  1  0
  1  1  1  0  1  1  1  0  0
  1  *  1  0  0  0  0  0  0
  1  1  1  0  0  0  0  0  0

  % java Minesweeper 16 16 40
  1  1  1  1  *  1  0  0  0  0  0  1  1  1  0  0
  2  *  4  3  3  2  2  1  1  0  0  1  *  2  1  0
  2  *  *  *  3  *  3  *  1  1  1  2  3  *  2  0
  1  2  4  *  3  3  *  3  1  1  *  1  2  *  2  0
  0  0  1  1  1  2  *  3  1  1  2  3  3  3  2  1
  0  0  0  0  0  2  4  *  3  2  3  *  *  3  *  2
  0  0  0  0  1  3  *  *  4  *  *  3  2  3  *  2
  0  1  1  1  1  *  *  *  3  2  2  2  1  2  1  1
  0  2  *  2  1  2  3  2  1  0  0  2  *  3  1  1
  1  3  *  2  0  0  1  2  3  2  1  3  *  4  *  1
  1  *  2  1  0  0  1  *  *  *  1  2  *  3  1  1
  1  1  1  1  1  1  1  2  3  2  1  1  2  2  1  0
  0  0  0  1  *  2  1  1  0  0  0  0  1  *  1  0
  0  0  0  1  1  2  *  1  0  0  1  1  2  1  1  0
  0  0  0  0  0  1  1  1  0  0  1  *  1  0  0  0
  0  0  0  0  0  0  0  0  0  0  1  1  1  0  0  0

  % java Minesweeper 16 30 82
  1  1  1  0  1  *  2  1  0  0  0  0  1  1  1  0  0  0  1  1  2  *  *  3  1  1  0  0  1  1
  1  *  1  0  1  2  *  1  0  0  0  0  1  *  2  1  0  0  1  *  2  3  *  3  *  2  2  1  2  *
  1  1  1  0  1  2  2  1  0  0  1  1  2  3  *  3  1  1  1  1  1  1  1  2  2  *  2  *  2  1
  0  0  0  0  1  *  2  1  1  0  1  *  1  3  *  4  *  1  0  1  1  1  0  0  2  2  3  1  1  0
  0  0  1  2  3  2  3  *  3  2  2  2  1  2  *  3  2  2  1  1  *  2  1  0  1  *  2  2  1  1
  0  0  2  *  *  1  2  *  *  2  *  1  0  2  2  2  1  *  1  1  2  *  1  0  2  4  *  3  *  2
  0  0  2  *  5  3  3  3  3  2  1  1  0  1  *  3  3  2  1  0  1  1  1  1  2  *  *  4  3  *
  0  1  2  4  *  *  2  *  1  0  0  0  0  2  3  *  *  2  1  2  1  2  1  2  *  3  2  2  *  2
  1  2  *  3  *  3  2  1  1  0  0  0  1  3  *  4  2  3  *  4  *  2  *  2  1  1  0  1  1  1
  *  2  1  2  1  1  0  0  0  1  2  2  2  *  *  2  0  2  *  *  3  3  2  1  0  0  0  0  0  0
  2  2  0  0  0  0  1  1  1  1  *  *  3  3  3  1  0  1  2  2  2  *  1  0  0  1  1  1  0  0
  *  2  0  0  0  0  1  *  2  2  2  2  2  *  2  1  0  1  1  1  1  1  1  1  2  *  2  1  0
  *  2  0  0  0  0  1  2  *  1  1  1  3  3  *  1  1  2  *  1  0  0  0  1  *  4  4  *  1  0
  1  1  1  1  1  1  1  2  1  1  1  *  3  *  2  2  2  *  3  2  2  1  1  1  2  *  *  3  2  1
  1  1  2  *  1  1  *  1  0  0  2  3  *  2  1  1  *  2  2  *  2  *  1  0  1  3  3  3  *  2
  1  *  2  1  1  1  1  1  0  0  1  *  2  1  0  1  1  1  1  1  2  1  1  0  0  1  *  2  2  *

  % java Minesweeper 4 8 0
  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0

  % java Minesweeper 8 4 32
  *  *  *  *
  *  *  *  *
  *  *  *  *
  *  *  *  *
  *  *  *  *
  *  *  *  *
  *  *  *  *
  *  *  *  *

  % java Minesweeper 1 20 10
  0  0  0  0  1  *  *  2  *  *  *  2  *  *  2  *  *  *  1  0
```

```
==> passed

Test 2: check that counts are consistent with mines (varying k)
  * m =  4, n =  8, k random [1000 trials]
  * m =  8, n =  4, k random [1000 trials]
  * m =  5, n = 40, k random [1000 trials]
  * m =  7, n = 30, k random [1000 trials]
  * m = 10, n = 10, k random [1000 trials]
==> passed

Test 3: check that counts are consistent with mines (fixed k)
  * k =  1, m and n random   [1000 trials]
  * k = 10, m and n random   [1000 trials]
  * k = 20, m and n random   [1000 trials]
  * k = 50, m and n random   [1000 trials]
  * k = 80, m and n random   [1000 trials]
  * k = 90, m and n random   [1000 trials]
  * k = 99, m and n random   [1000 trials]
==> passed

Test 4: check that counts are consistent with mines (corner cases)
  * m =  5, n = 10, k =  0
  * m = 10, n =  5, k =  0
  * m =  5, n = 10, k = 50
  * m = 10, n =  5, k = 50
  * k =  0, m and n random   [1000 trials]
  * k =  1, m and n random   [1000 trials]
==> passed

Test 5: check that program produces different results each time
  * m =  4, n =  8, k = 16 [2 trials]
  * m =  8, n =  4, k = 26 [2 trials]
  * m =  1, n = 20, k = 16 [2 trials]
  * m = 20, n =  1, k = 10 [2 trials]
==> passed

Test 6: check number of mines, with k varying
  * m =  4, n =  8, k random [1000 trials]
  * m =  8, n =  4, k random [1000 trials]
  * m =  5, n = 40, k random [1000 trials]
  * m =  7, n = 30, k random [1000 trials]
  * m = 10, n = 10, k random [1000 trials]
==> passed

Test 7: check number of mines, with k fixed
  * k =  5, m and n random   [1000 trials]
  * k = 10, m and n random   [1000 trials]
  * k = 50, m and n random   [1000 trials]
  * k = 99, m and n random   [1000 trials]
==> passed

Test 8: check number of mines for corner cases
  * m =  5, n = 20, k =  0
  * m = 20, n =  5, k =  0
  * m =  5, n = 10, k = 50
  * m = 10, n =  5, k = 50
  * k =  0, m and n random   [1000 trials]
  * k =  1, m and n random   [1000 trials]
==> passed

Test 9: check that mines are uniformly random
  * m = 1, n = 2, k = 1 [repeated 15000 times]
  * m = 1, n = 3, k = 1 [repeated 15000 times]
          value  observed  expected   2*O*ln(O/E)
       ------------------------------------------
            1-1      3746    5000.0      -2163.31
            1-2      7448    5000.0       5936.17
            1-3      3806    5000.0      -2077.00
       ------------------------------------------
                    15000   15000.0       1695.86

    G-statistic = 1695.86 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

  * m = 2, n = 2, k = 2 [repeated 15000 times]
  * m = 2, n = 4, k = 3 [repeated 15000 times]
          value  observed  expected   2*O*ln(O/E)
       ------------------------------------------
            1-1      4078    5625.0      -2623.09
            1-2      7233    5625.0       3637.23
            1-3      7268    5625.0       3725.00
            1-4      3965    5625.0      -2773.24
            2-1      4040    5625.0      -2674.29
            2-2      7171    5625.0       3482.58
            2-3      7179    5625.0       3502.48
            2-4      4066    5625.0      -2639.33
       ------------------------------------------
                    45000   45000.0       3637.34

    G-statistic = 3637.34 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

  * m = 3, n = 3, k = 6 [repeated 15000 times]
          value  observed  expected   2*O*ln(O/E)
       ------------------------------------------
            1-1      7618   10000.0      -4145.28
            1-2     11349   10000.0       2872.31
            1-3      7521   10000.0      -4285.25
            2-1     11409   10000.0       3007.81
            2-2     13992   10000.0       9399.84
            2-3     11376   10000.0       2933.21
            3-1      7686   10000.0      -4045.67
            3-2     11437   10000.0       3071.26
            3-3      7612   10000.0      -4154.01
```

```
        -----------------------------------------
                     90000   90000.0       4654.21

    G-statistic = 4654.21 (p-value = 0.000000, reject if p-value <= 0.0001)
    Note: a correct solution will fail this test by bad luck 1 time in 10,000.

==> FAILED

Test 10: check statistical independence of mines within an m-by-n grid
  * m = 500, n = 500, k = 125000
  * m = 500, n = 500, k = 25000
  * m = 500, n = 500, k = 225000
  * m = 100, n = 900, k = 27000
  * m = 900, n = 100, k = 63000
==> passed

Test 11: check statistical independence of mines between m-by-n grids
  * m = 1, n = 2, k = 1 [repeated 50000 times]
  * m = 1, n = 3, k = 1 [repeated 50000 times]
  * m = 2, n = 2, k = 2 [repeated 50000 times]
  * m = 2, n = 4, k = 3 [repeated 50000 times]
  * m = 3, n = 3, k = 8 [repeated 50000 times]
==> passed


Minesweeper Total: 10/11 tests passed!


================================================================
```